DECENTRALIZED AUTHENTICATION · DEMOS/06-OPERATOR-TRUST-AND-CREDENTIALS

One Operator, No User List, One Push

Eight patterns measured on one bare NATS server in operator mode, driven by the `nats auth` CLI. Each card states the mechanism, what it buys, what it costs, and what the server actually did. One claim in here was wrong first, and the correction travels with it.

THE QUESTION EVERY CARD SERVES

Can a NATS server admit a user it has **never seen by name**, only through a signed operator → account → user chain — and then **refuse that user** when its credential expires or is revoked?

THE RULE THAT ORDERS THE DECK

The server trusts **one operator key** and stores **accounts**, never users. So anything that lives in a user JWT works with no server change, and anything that lives in an account JWT **does nothing until it is pushed**.

THE EIGHT PATTERNS

- 01 · The server trusts one key, not a user list
- 02 · An account must be pushed before its users get in
- 03 · A push report is not evidence of trust
- 04 · Permissions travel inside the user JWT
- 05 · A scoped signing key: the role wins
- 06 · Expiry ends access without a push
- 07 · A revocation is only as fast as its push
- 08 · The client cannot tell you why it was refused

01

TRUST

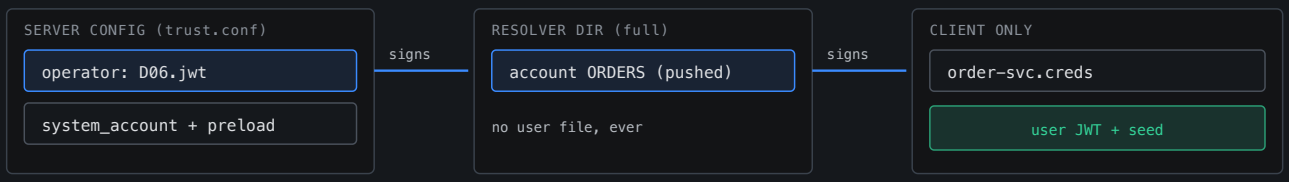
The server trusts one key, not a user list

In operator mode the server config names **one operator JWT** and the `SYSTEM` account. The operator signs accounts; an account signs users. A user gets in by showing its JWT and signing a nonce with its own seed. The server checks the chain, not a list.

DECISION

Does the server need to know each user? **No.** It needs the operator key and the account JWTs. Users are issued, changed and dropped without touching the server's config.

MECHANISM



- **No user in the config, none on disk.** `.run/trust.conf` holds three settings: `operator`, `system_account`, `resolver_preload` (`D06-R1`). The resolver directory never held a `U…` file (`D06-R4`, check C16).
- **A user issued while the server runs gets in** — no push, no restart (check P2). The server PID did not change through all of exercise 02 (P13).
- **The seed proves the holder.** The right `order-svc` JWT with another user's seed was refused (C12). The server log names users by key (`jwt:U…`), never by name (P5–P6).

WHAT IT BUYS

- **The server config stops changing** when people and services come and go.
- **Delegation.** Whoever holds an account key can issue users for that account, and only that one.
- **No user secret on the server.** From the docs, not measured: the seed never leaves the client.

WHAT IT COSTS

- **Keys become your problem.** Every seed sits in a key store (mode 600, measured). Lose the operator seed and you rebuild the chain.
- `nats auth … --json` **prints seeds** (measured). Never paste it anywhere.
- **Logs name keys, not people.** Tracing a refusal back to a user needs a key-to-name map.

VERDICT

Use operator mode when users change faster than you want to edit server config. Treat the **operator and account seeds** as the real secrets — they are now the user list.

02

TRUST

An account must be pushed before its users get in

With the `full` resolver the server learns an account only when its JWT is **pushed** by a `SYSTEM` user. Until then every user of that account is refused, however valid its own JWT is.

DECISION

Is a push a deploy step or an optional sync? **A deploy step.** Make it part of the pipeline, and check its result on the server, not in the push output.

MEASURED · 2026-10-08 · NATS-SERVER V2.14.6 ON THE AUTHOR'S MAC · CHECKS C1–C3, Q1–Q2, P9–P10

WHAT WAS TRIED	CLIENT	SERVER
`order-svc`, account **not** pushed	Authorization Violation	account fetch ~**1.9 s**, `fetching jwt timed out`, `authentication error`
`auth account query ORDERS`, before the push	did not receive a valid token	no copy to return
`auth account push ORDERS` as `SYSTEM` `admin`	success	JWT written to the resolver directory
`order-svc`, after the push	published, exit 0	nothing logged; message delivered
user of a **new scoped key**, before the push	Authorization Violation	`authentication error` (no fetch: the account was known)

- **What needs a push:** a new account, and any change to an account JWT — here a new signing key and a revocation (card 07). **What does not:** a new user signed by a key the server already has.
- **Two copies, two counts.** After the push the server copy said `Users: 0` and the local one `Users: 1`. That is normal: the server stores no user JWTs.

WHAT IT BUYS

- **One gate per account.** Nothing in an account works until somebody with `SYSTEM` rights publishes it.
- **A readable server copy.** `auth account query` shows what the server holds, so drift is visible.

WHAT IT COSTS

- **A forgotten push looks like bad credentials.** The client prints the same text either way.
- **An unknown account costs a ~1.9 s fetch** before the refusal — per attempt.
- **Local and server copies drift** silently until you query.

VERDICT

Push every account change, then confirm with `auth account query`. A user that "suddenly cannot connect" after an account edit is a missing push first.

03

TRUST

A push report is not evidence of trust

A second, throwaway operator `ROGUE` made an account and a user with the **same names**: `ORDERS` and `order-svc`. The server never trusted it. The CLI still reported the rogue push as a success.

DECISION

How do you know the server accepted an account? **Ask the server** — `/accountz` or `account query`. The push output and the resolver directory both say yes to an account the server refused.

MEASURED · 2026-10-08 · CHECKS C6–C11 · PREDICTION FOR C7 WAS WRONG

EVIDENCE	SAYS	TRUE?
CLI output of the rogue `account push`	`Success 1 Failed 0`	**No**
`.run/resolver/` directory	the `ROGUE` account JWT is there	**No**
`/accountz`	`ORDERS`, `SYSTEM`, `$G` only	**Yes**
the rogue user tries to connect	refused, `authentication error`	**Yes**

WHAT HOLDS

- **The trust check is real.** Same names, wrong operator: refused, before and after its push.
- **The server log does not lie:** the rogue push logged nothing; the refusals logged one line each.

WHAT MISLEADS

- **The push says "success"** and the file lands on disk. Neither is the server's decision.
- **An open question:** in one run of six, one refusal logged three lines, not one. Not explained.

RETRACTION — "A FOREIGN OPERATOR IS REFUSED WITH NO FETCH" WAS TOO WIDE

The first write-up said the two refusals differ in the server log: an unpushed account waits ~1.9 s for a fetch, a foreign operator's user does not. The hand walkthrough on 2026-10-08 showed that holds **only after the rogue account was pushed**. Before its push, the rogue user waited the same ~1.9 s fetch as an unpushed `ORDERS` user. The server refuses an **unknown account key** first, and only reaches the operator check once it holds a JWT for that key. The lesson survived; its scope shrank.

VERDICT

Verify an account on the server, by key, after every push. Never treat a CLI success line or a file in the resolver directory as proof of trust.

04

PERMISSIONS

Permissions travel inside the user JWT

`analytics-reader` was issued with `--sub-allow 'orders.>'` and `--pub-deny '>'` while the server ran. Those rules were written into its signed JWT, and the server enforced them on the first connection — with no push and no restart.

DECISION

Where should a user's permissions live? **In its JWT**, signed by the account. The server reads them at connect time; nobody edits server config.

MEASURED · 2026-10-08 · CHECKS P1–P7, P4B · `D06-R5`

auth user add … --sub-allow orders.> --pub-deny > → JWT: pub deny [>], sub allow [orders.>] → connect: admitted → publish: denied

ACTION	CLIENT	SERVER LOG
subscribe `orders.>`	receives `order-svc` 's message	nothing
publish `orders.created`	Permissions Violation for Publish	`jwt:U…" - Publish Violation`
subscribe `invoices.>`	Permissions Violation for Subscription	`jwt:U…" - Subscription Violation`

- **Authentication and authorization are separate results.** The reader was admitted, then each forbidden action was denied on its own. The connection stayed up.
- **Read the whole client output.** The CLI printed `Subscribing on invoices.>` before the violation. The first line does not mean the server allowed it.

WHAT IT BUYS

- **A new role is a new credential,** issued by the account owner with no server change.
- **The rules are signed,** so the holder cannot widen them.
- **Each denial is logged** with the user's key and the subject.

WHAT IT COSTS

- **Changing a user's rules means a new JWT.** Not measured here: the old JWT keeps its old rules until it expires or is revoked (cards 06, 07).
- **Every user carries its own copy** of the rules. Many users with one role means many copies — card 05 is the fix.

VERDICT

Put per-user rules in the user JWT and pair them with a short lifetime, so a rule change reaches every holder within one expiry.

05

PERMISSIONS

A scoped signing key: the role wins

A **scoped signing key** is an account key with a permission template attached. Here the key `reader` carries "subscribe `orders.>`, publish nothing". A user signed with it gets the **key's** rules, whatever the user asked for.

DECISION

One role, many users — where should the rules live? **On a scoped key in the account JWT.** The user JWT then carries no rules of its own, and cannot be issued with wider ones.

MEASURED · 2026-10-08 · CHECKS P8–P11, P9B, P11B · `D06–R6`

STEP	RESULT
`greedy-reader` issued with `--key reader --pub-allow '>'`	exit 0, but the JWT's `pub` is `{}`. **The CLI dropped the request** and printed the role.
connect, before the account push	refused — the server did not know the new key yet
connect, after the push	admitted, receives `orders.>`
publish `orders.created`	denied by the role; server logs `jwt:<greedy key>" - Publish Violation`

- **From the source, not measured:** at login the server replaces the user's whole permission and limit set with the key's template (nats-server v2.14.6 `auth.go`).
- **Payload limits are not settled.** The CLI showed the role's payload as `0`, then as `unlimited`; the source says both mean "no role limit". A probe with an explicit 100-byte limit was **inconclusive**: the client refused the 200-byte message itself, and the server logged nothing.

WHAT IT BUYS

- **A role defined once,** in the account JWT, instead of copied into every user.
- **No privilege creep at issue time.** The CLI would not write wider rules than the role.

WHAT IT COSTS

- **A new key needs an account push** before its users get in (card 02).
- **The drop is silent.** `--pub-allow '>'` exits 0 and does nothing. Read the printed rules.
- **Limits on a role are not proven here** — only permissions are.

VERDICT

Use one scoped key per role for services and readers. Keep per-user JWT rules for the rare user who does not fit a role.

06

LIFETIME

Expiry ends access without a push

`--expire` writes an end time (`exp`) into the user JWT. After it, the server refuses the credential. Nothing is pushed and nothing on the server changes: the end time travels with the credential.

DECISION

Does an expiry also end an **open** connection, or only new ones? **Both.** The open connection was cut within one second of `exp` .

MEASURED · 2026-10-08 AND 2026-10-09 · CHECKS E1–E6, E4B · `D06-R7`



TIME → the cut listener's exit code at the end: 0 (E6)

- **On time.** Script runs: cut within 1 s of `exp` . Hand run: `exp` 11:53:29, cut 11:53:29.
- **The server logs nothing at the cut.** The log shows only the refused reconnects after it.
- **The trap:** `nats sub --wait` printed `Disconnected due to: EOF` , kept reconnecting, and **exited 0**. A script that trusts the exit code misses the cut.

WHAT IT BUYS

- **Access that ends by itself,** on every connection, with no server action.
- **A hard bound on a leaked credential:** at most one lifetime.

WHAT IT COSTS

- **Somebody must re-issue before the end,** and get the new file to the client in time.
- **Clients see a plain disconnect.** Nothing in the client output says "expired".
- **Clock trust:** the end time is absolute; skewed clocks shift it (not tested).

VERDICT

Give service credentials a lifetime you can re-issue inside, and watch connections, not exit codes, to see a credential end.

07

LIFETIME

A revocation is only as fast as its push

The server stores no users, so it cannot delete one. A revocation is a line in the **account** JWT: "stop trusting this user key". `auth user rm --revoke` writes that line on your disk. The server learns it only at `auth account push`.

DECISION

How fast can you cut off one user? **As fast as you push.** After the push, the open connection was cut within a second. Before it, nothing changed at all.

MEASURED · 2026-10-08 AND 2026-10-09 · CHECKS Q1–Q2, R1–R6, R4B · `D06–R8`

STATE	LOCAL COPY	SERVER COPY	NEW CONNECTION	OPEN CONNECTION
Revoked, not pushed	`Revocations: 1`	`Revocations: 0`	still admitted	still receives
Pushed	`Revocations: 1`	`Revocations: 1`	refused	cut within 1 s

- **Hand run, server side:** account updated 21:36:07.69, first refused reconnect 21:36:08.49 — about **0.8 s**.
- **Only that user.** `order-svc`, in the same account, kept publishing (R5). `/accountz` lists the reader's key under `revoked_user` (R6).
- **The recorded time is the `rm` time,** not the push time. The server acted only at the push.
- **Not tested:** a JWT for the same key signed *after* the revocation time may be admitted; `auth user rm` without `--revoke` only deletes the local copy. Both are from the source, not this lab. The rule "an account change waits for its push" was measured for a revocation only.

WHAT IT BUYS

- **An off switch for one user** before its credential expires.
- **Fast once pushed:** open and new connections both end within a second.

WHAT IT COSTS

- **A revocation without a push is a false sense of safety.** The CLI says done; the user is still in.
- **The account JWT grows** with every revoked key.
- **Pushing needs `SYSTEM` rights,** so the off switch has its own credential to guard.

VERDICT

Treat "revoke" and "push" as one action, and confirm it with `account query`. Use short expiry as the default way access ends; keep revocation for the emergency.

08

EVIDENCE

The client cannot tell you why it was refused

Six different causes gave the client the same line: `Authorization Violation`, exit 1. The server log tells some of them apart, and at its default level it gives no reason at all. A refusal is only evidence next to a positive control and its server line.

DECISION

What do you record for a refusal? **Both sides:** the client's text and exit code, and the server's log line — next to a valid user that got through on the same server, in the same run.

MEASURED · 2026-10-08 AND 2026-10-09 · EXERCISES 01–03

CAUSE	CLIENT	SERVER LOG (DEFAULT LEVEL)
account not pushed	Authorization Violation	~1.9 s fetch, timed out, `authentication error`
foreign operator, account not pushed	Authorization Violation	the same fetch, then `authentication error`
foreign operator, account pushed	Authorization Violation	`authentication error` only
no credentials · wrong seed	Authorization Violation	`authentication error` only
expired · revoked and pushed	Authorization Violation	`authentication error` only
a forbidden publish or subscribe	Permissions Violation for …	`jwt:U…" – Publish / Subscription Violation`

- **One refused connect, one line.** Each refusal checked (P9b, E4b, R4b) added exactly one `authentication error`.
- **A cut client keeps knocking.** A cut listener retried about every 2 s, one line per try — 16 lines in one 2026-10-09 run. Count a refusal only after it has stopped.
- **From the source, not tried:** the reason exists only at debug level.

WHAT DOES TELL YOU

- **The fetch line** splits "unknown account" from the rest.
- `/accountz`, `/connz?auth=1`, `account query` show what the server holds, by key.
- **Violation lines** name the user key and the subject.

WHAT DOES NOT

- **The client's error text** — one line for six causes.
- **The exit code** of a cut subscriber (0) and of a push (success for a refused account).
- **The boot log's** `System : ""` — `/varz` showed the system account was set from the operator JWT.

VERDICT

Debug auth from the server side. Give operators the monitor endpoints and the log, and give users one runbook line: "Authorization Violation means ask, it does not say why".

?

QUICK REFERENCE

Which card answers your question

Start from the thing you are trying to decide, not from the command you were reading about.

YOU ARE ASKING…	CARD	SHORT ANSWER
Does the server need a user list?	**01** The server trusts one key, not a user list	No. One operator key; users are checked by chain.
Why is a valid user refused?	**02** An account must be pushed before its users get in	Check the push first. Then ask the server log.
Did the server accept my account?	**03** A push report is not evidence of trust	Ask `/accountz` or `account query`, not the push output.
Where do a user's permissions live?	**04** Permissions travel inside the user JWT	In its signed JWT. No server change, no push.
Many users, one role?	**05** A scoped signing key: the role wins	A scoped key. One push, and the role beats the request.
How should access end by default?	**06** Expiry ends access without a push	Short lifetimes. Open connections are cut too.
I must cut one user off now.	**07** A revocation is only as fast as its push	Revoke **and** push. Then confirm.
What does this refusal mean?	**08** The client cannot tell you why it was refused	Read the server log; compare with a positive control.

DEFAULTS WORTH COPYING

- **Run every `nats` call with `--no-context`** and its own key store (`XDG_DATA_HOME`). Without it, `nats auth` writes into your real store.
- **Push, then query.** Every account change ends with `auth account query` .
- **Bind the server to loopback** for a lab. One server per port; stop it by PID, never `pkill nats-server` .

TRAPS THIS DEMO WALKED INTO

- **A rogue push said "success".** Card 03.
- **`--json` printed seeds.** Never use it in a walkthrough or a ticket.
- **A cut subscriber exited 0,** and a cut listener's retries inflated the error count.
- **`auth user credential` refuses to overwrite** a file without `--force` .

VERDICT

User JWTs change without the server; account JWTs change only with a push. Everything else in this deck follows from that split.

§

METHOD

Where every number came from

A figure with no date and no machine becomes a constant, and a constant goes stale quietly. Each number in this deck is listed with the day it was taken and the thing it ran on.

THE MACHINE

- **nats-server v2.14.6**, one bare process on the author's Mac (Darwin 25.4.0), loopback `127.0.0.1:4922`, monitor `8922`. Operator mode, `full` resolver. No Docker, no cluster, no JetStream, no TLS.
- `nats` **CLI 0.4.0** (`nats auth`; no `nsc`), always with `--no-context` and a key store inside the demo's `.run/`.
- One operator `D06`, one account `ORDERS` plus `SYSTEM`; a throwaway `ROGUE` operator only to be refused.

THE NUMBERS

CARD	FIGURE	TAKEN	ON WHAT
02, 03, 08	account fetch ~**1.9 s** before an unknown-account refusal (1.901 s in the walkthrough)	2026-10-08	ex01 C1–C2 and C6, script and hand runs
03	one refusal logged 3 lines in 1 run of 6; 1 line in the other 5	2026-10-08	ex01 C17, open question
06	open connection cut within **1 s** of `exp`; hand run 11:53:29 / 11:53:29	2026-10-08, -09	ex03 E3, 10 s credential (script), 60 s (hand)
07	open connection cut within **1 s** of the push; hand run ~**0.8 s**	2026-10-08, -09	ex03 R3; `/accountz` update_time vs first refused reconnect
08	retries about every **2 s**; 16 `authentication error` lines in one run	2026-10-09	saved ex03 server log, two cut listeners
08	exactly **1** log line per refused connect	2026-10-09	checks P9b, E4b, R4b
01	user `Max Payload` 1 048 576 from `--defaults` (not enforced in a test)	2026-10-08	`auth user info order-svc`
05	100-byte scoped limit: 50 B delivered, 200 B refused by the client	2026-10-09	optional payload probe, by hand, inconclusive

HOW TO READ THEM

- **Read the behaviour, not the value.** "Cut within a second of the push" transfers; "0.8 s" is one laptop on one evening.
- **Every check script re-runs from nothing:** `exercises/ex01-check.sh`, `ex02-check.sh`, `ex03-check.sh`. All three printed ALL PASS on 2026-10-09.
- **One claim here has already been narrowed.** The retraction on card 03 is kept beside the card it corrected.
- **"From the source"** means read in the code of the installed versions, not observed. Those lines are labelled on each card.
- **Not yet done:** `D06-R9`, a walkthrough by another person with no help.

Sources in the repo: `demos/06-operator-trust-and-credentials/README.md` · `exercises/EXERCISE_OBSERVATIONS.md` · `exercises/EXERCISE-0{1,2,3}-TERMINAL-STEPS.md` · `exercises/ex0{1,2,3}-check.sh`. Docs: docs.nats.io/learn/security/operator-mode and /decentralized-auth.